

# Tugas Besar 2

## Pasti-Gacor-Zeus

IF2211 Strategi Algoritma

Implementasi Pattern Matching pada Chromium Browser Extension  
untuk Deteksi Judi Online

Semester 2 Tahun 2025/2026



**Disusun oleh:**

**Pasti-Gacor-Zeus**

|                           |            |
|---------------------------|------------|
| Athilla Zaidan Zidna Fann | (13524068) |
| Stevanus Agustav Wongso   | (13524020) |
| Ray Owen Martin           | (13524033) |

Laboratorium Ilmu dan Rekayasa Komputasi  
Program Studi Teknik Informatika  
Sekolah Teknik Elektro dan Informatika  
Institut Teknologi Bandung

# Daftar Isi

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Pendahuluan</b>                                  | <b>2</b>  |
| 1.1      | Kisah . . . . .                                     | 2         |
| 1.2      | Latar Belakang . . . . .                            | 3         |
| 1.3      | Deskripsi Tugas . . . . .                           | 3         |
| 1.3.1    | Masukan . . . . .                                   | 4         |
| 1.3.2    | Keluaran . . . . .                                  | 4         |
| <b>2</b> | <b>Dasar Teori</b>                                  | <b>6</b>  |
| 2.1      | Ekstensi Browser Chromium . . . . .                 | 6         |
| 2.2      | Document Object Model . . . . .                     | 6         |
| 2.3      | Pencocokan String . . . . .                         | 7         |
| 2.4      | Regular Expression . . . . .                        | 7         |
| 2.5      | Levenshtein Distance . . . . .                      | 8         |
| 2.5.1    | Weighted Levenshtein Distance . . . . .             | 8         |
| 2.6      | Boyer-Moore . . . . .                               | 8         |
| 2.6.1    | Algoritma Rabin-Karp . . . . .                      | 9         |
| 2.7      | Tokenisasi Teks . . . . .                           | 9         |
| 2.8      | Deduplication dan Cache . . . . .                   | 10        |
| 2.9      | Optical Character Recognition . . . . .             | 10        |
| 2.10     | Highlight, Tooltip, dan Blur . . . . .              | 10        |
| 2.11     | Kompleksitas Algoritma . . . . .                    | 11        |
| <b>3</b> | <b>Analisis Pemecahan Masalah</b>                   | <b>12</b> |
| 3.1      | Langkah-Langkah Pemecahan Masalah . . . . .         | 12        |
| 3.2      | Pemetaan Masalah menjadi Elemen Algoritma . . . . . | 12        |
| 3.3      | Fitur Fungsional dan Arsitektur Extension . . . . . | 14        |
| 3.3.1    | Fitur Fungsional . . . . .                          | 14        |
| 3.3.2    | Arsitektur Extension . . . . .                      | 14        |
| 3.4      | Contoh Ilustrasi Kasus . . . . .                    | 15        |
| <b>4</b> | <b>Implementasi dan pengujian</b>                   | <b>16</b> |
| <b>5</b> | <b>Penutup</b>                                      | <b>17</b> |
| <b>6</b> | <b>Lampiran</b>                                     | <b>18</b> |

# Bab 1

## Pendahuluan

### 1.1 Kisah

Perkembangan web membuat informasi dapat disebar dengan sangat cepat. Halaman web, media sosial, forum, dan situs promosi dapat memuat berbagai jenis konten hanya dalam hitungan detik. Di sisi lain, kemudahan penyebaran informasi juga membuka ruang bagi munculnya konten berisiko, salah satunya konten yang berkaitan dengan judi daring atau *judi online* (judol). Konten seperti ini sering kali muncul dalam bentuk kata kunci eksplisit, variasi penulisan, angka yang disisipkan ke dalam kata, hingga teks yang tertanam di dalam gambar.

Deteksi manual terhadap konten judol tidak selalu praktis. Sebuah halaman web dapat memiliki banyak elemen teks, tautan, gambar promosi, dan istilah yang sengaja dimodifikasi agar tidak mudah ditemukan oleh pencarian biasa. Sebagai contoh, kata seperti **gacor** dapat ditulis menjadi **g4cor**, **slot** dapat muncul sebagai bagian dari frasa promosi, dan nama situs dapat ditulis dengan pola huruf diikuti angka seperti **ZEUS222**. Variasi semacam ini membuat pencocokan kata secara eksak saja tidak cukup.

Permasalahan tersebut dapat dipandang sebagai persoalan pencocokan string. Sebuah sistem perlu membaca teks pada halaman web, membandingkannya dengan daftar kata kunci yang dicurigai, mengenali pola tertentu menggunakan *regular expression*, serta mendeteksi bentuk penulisan yang mirip melalui pendekatan *approximate string matching*. Selain itu, karena sebagian konten promosi muncul dalam gambar, sistem juga perlu memiliki kemampuan *Optical Character Recognition* (OCR) agar teks pada gambar dapat diproses oleh pipeline deteksi yang sama.

Pada Tugas Besar ini, dikembangkan sebuah ekstensi Chromium bernama **Judol Detector**. Ekstensi ini melakukan pemindaian terhadap teks yang terlihat pada halaman web, mendeteksi indikasi konten judol menggunakan kombinasi algoritma pencocokan string, menandai teks yang terdeteksi secara langsung pada halaman, serta menampilkan statistik hasil deteksi melalui popup ekstensi. Program juga menyediakan mode blur untuk menyamarkan konten yang terdeteksi dan fitur OCR untuk memindai teks yang tertanam pada gambar.

## 1.2 Latar Belakang

Pencocokan string merupakan salah satu persoalan mendasar dalam ilmu komputer. Dalam bentuk paling sederhana, pencocokan string bertujuan menemukan kemunculan suatu pola dalam teks. Persoalan ini banyak digunakan dalam mesin pencari, editor teks, bioinformatika, keamanan siber, dan sistem moderasi konten. Algoritma klasik seperti Knuth-Morris-Pratt (KMP) dan Boyer-Moore dirancang untuk melakukan pencocokan eksak secara efisien dengan mengurangi jumlah perbandingan karakter yang tidak perlu [1].

Pada konteks deteksi konten judol, pencocokan eksak berguna untuk menemukan kata atau frasa yang sudah diketahui, seperti `slot`, `rtp`, `bonus deposit`, atau `situs judi`. Namun, konten promosi sering menggunakan variasi penulisan untuk menghindari deteksi sederhana. Penulisan seperti `H0KI88`, `G4C0R`, atau `SLOT777` tetap dapat dimaknai oleh manusia sebagai istilah judol, meskipun tidak sama persis dengan kata kunci dasar. Oleh karena itu, diperlukan pendekatan lain seperti *regular expression* dan *fuzzy matching*.

*Regular expression* dapat digunakan untuk mendeteksi pola tekstual yang terstruktur, misalnya gabungan huruf dan angka yang sering digunakan pada nama situs atau kode promosi [2]. Sementara itu, *Levenshtein Distance* mengukur jarak edit antara dua string berdasarkan jumlah operasi penyisipan, penghapusan, dan penggantian karakter yang diperlukan untuk mengubah satu string menjadi string lain [3]. Dengan memberikan bobot khusus pada karakter yang mirip secara visual, sistem dapat mengenali variasi seperti huruf `o` yang diganti angka `0`, huruf `a` yang diganti angka `4`, atau huruf `s` yang diganti angka `5`.

Web modern juga tidak hanya memuat teks dalam bentuk node DOM. Banyak konten promosi disisipkan dalam gambar, banner, atau elemen visual. Untuk menangani kasus tersebut, Judol Detector menggunakan OCR berbasis Tesseract.js untuk mengambil teks dari gambar yang terlihat pada halaman [4]. Teks hasil OCR kemudian diproses menggunakan pipeline deteksi yang sama dengan teks DOM.

Pemilihan bentuk ekstensi browser memungkinkan deteksi dilakukan langsung pada halaman yang sedang dibuka pengguna. Dengan Manifest V3, ekstensi Chromium dapat menjalankan *content script* untuk berinteraksi dengan halaman, serta menyediakan popup sebagai antarmuka ringkas untuk menampilkan status dan statistik pemindaian [5]. Pendekatan ini membuat sistem tidak perlu mengunduh halaman melalui server terpisah, melainkan bekerja di sisi klien pada halaman yang sedang aktif.

## 1.3 Deskripsi Tugas

Program Judol Detector adalah ekstensi Chromium untuk mendeteksi indikasi konten judi daring pada halaman web. Program memindai teks yang terlihat pada halaman, menjalankan algoritma pencocokan terhadap daftar kata kunci, menandai hasil deteksi pada halaman, dan menampilkan ringkasan statistik melalui popup.

Secara umum, alur kerja program adalah sebagai berikut:

1. Pengguna membuka halaman web yang ingin diperiksa.

2. Pengguna membuka popup ekstensi dan menjalankan proses pemindaian.
3. *Content script* mengumpulkan node teks yang terlihat pada DOM.
4. Program menjalankan pipeline deteksi, yaitu pencocokan pola dengan RegEx dan pencocokan fuzzy dengan Weighted Levenshtein.
5. Jika OCR diaktifkan, program juga memindai gambar yang terlihat, melakukan preprocessing citra, dan menjalankan pipeline deteksi terhadap teks hasil OCR.
6. Teks atau gambar yang terdeteksi diberi penanda visual pada halaman.
7. Popup menampilkan jumlah hasil deteksi, waktu eksekusi, perbandingan algoritma, kata kunci paling sering muncul, dan ringkasan OCR.

Program dikembangkan menggunakan TypeScript, React, Vite, Tailwind CSS, dan Tesseract.js. TypeScript digunakan untuk menjaga struktur data deteksi, React digunakan untuk membangun antarmuka popup, Vite digunakan sebagai build tool, Tailwind CSS digunakan untuk styling, dan Tesseract.js digunakan untuk fitur OCR.

### 1.3.1 Masukan

Masukan yang digunakan oleh program terdiri atas beberapa komponen berikut:

1. **Halaman web aktif**, yaitu halaman yang sedang dibuka pada browser Chromium dan menjadi target pemindaian.
2. **Teks DOM yang terlihat**, yaitu kumpulan node teks yang dapat dibaca dari struktur DOM halaman dan tidak berada pada elemen yang harus diabaikan seperti `script`, `style`, `textarea`, dan `input`.
3. **Daftar kata kunci**, yaitu kumpulan istilah yang berkaitan dengan judul, seperti `slot`, `gacor`, `rtp`, `jackpot`, `casino online`, dan berbagai variasi frasa promosi lain.
4. **Gambar halaman**, yaitu elemen gambar yang terlihat pada viewport dan dapat diproses melalui OCR setelah melalui tahap preprocessing.
5. **Pengaturan popup**, yaitu pilihan pengguna seperti menjalankan ulang pemindaian, mengaktifkan OCR, dan mengaktifkan blur terhadap konten yang terdeteksi.

### 1.3.2 Keluaran

Keluaran yang dihasilkan program adalah sebagai berikut:

1. **Highlight pada halaman**, yaitu penandaan visual pada teks yang terdeteksi sebagai indikasi konten judul.



2. **Blur konten**, yaitu penyamaran visual terhadap teks atau gambar yang terdeteksi ketika fitur blur diaktifkan atau ketika gambar hasil OCR dinyatakan positif.
3. **Tooltip deteksi**, yaitu informasi tambahan yang muncul saat pengguna mengarahkan kursor ke hasil deteksi, berisi kata kunci, algoritma, jumlah kemunculan, dan waktu eksekusi.
4. **Ringkasan statistik**, yaitu jumlah total deteksi, waktu eksekusi, jumlah deteksi per algoritma, dan daftar kata kunci yang paling sering ditemukan.
5. **Ringkasan OCR**, yaitu jumlah gambar yang dipindai, jumlah gambar yang terdeteksi positif, jumlah gambar yang dilewati, dan waktu eksekusi OCR.
6. **Status pemindaian**, yaitu status seperti siap, sedang memindai, terdeteksi, bersih, atau gagal memindai.

## Bab 2

# Dasar Teori

Bagian ini menjelaskan teori yang menjadi dasar pengembangan Judol Detector. Konsep yang dibahas meliputi ekstensi browser, DOM, pencocokan string, *regular expression*, Weighted Levenshtein Distance, OCR, serta mekanisme penandaan hasil deteksi pada halaman web.

### 2.1 Ekstensi Browser Chromium

Ekstensi browser adalah perangkat lunak tambahan yang berjalan di dalam peramban untuk menambah atau memodifikasi perilaku halaman web. Pada Chromium, ekstensi didefinisikan melalui file `manifest.json` yang memuat metadata, permission, script, dan konfigurasi antarmuka [5].

Manifest V3 memisahkan beberapa komponen utama:

- **Popup**, yaitu antarmuka kecil yang muncul ketika pengguna menekan ikon ekstensi.
- **Content script**, yaitu script yang disisipkan ke halaman web dan dapat membaca atau memodifikasi DOM halaman.
- **Permission**, yaitu izin yang diperlukan ekstensi, misalnya izin membaca tab aktif, menyimpan data lokal, atau mengeksekusi script.

Pada Judol Detector, popup digunakan untuk menampilkan statistik dan kontrol pemindaian, sedangkan content script digunakan untuk mengambil teks halaman, menjalankan highlight, dan memberi efek blur pada konten yang terdeteksi.

### 2.2 Document Object Model

*Document Object Model* (DOM) adalah representasi struktur dokumen HTML dalam bentuk pohon hierarkis [6]. Setiap elemen HTML, teks, dan atribut dapat direpresentasikan sebagai node. Melalui DOM, script dapat membaca isi halaman, mencari elemen tertentu, dan memodifikasi tampilan halaman secara dinamis.

Dalam Judol Detector, DOM digunakan untuk dua keperluan utama. Pertama, content script menelusuri DOM untuk menemukan node teks yang terlihat dan

relevan untuk dipindai. Kedua, setelah sebuah teks terdeteksi, node teks tersebut diganti dengan elemen `span` khusus agar hasil deteksi dapat diberi highlight, tooltip, dan blur.

Tidak semua node teks perlu dipindai. Teks yang berada pada elemen seperti `script`, `style`, `noscript`, `textarea`, dan `input` diabaikan karena tidak merepresentasikan konten halaman yang umumnya dibaca pengguna. Pemfilteran ini membuat proses pemindaian lebih fokus pada teks yang benar-benar tampil pada halaman.

## 2.3 Pencocokan String

Pencocokan string adalah proses mencari kemunculan suatu pola dalam teks. Jika diberikan teks  $T$  dengan panjang  $n$  dan pola  $P$  dengan panjang  $m$ , tujuan pencocokan string adalah menemukan indeks pada  $T$  di mana  $P$  muncul. Persoalan ini merupakan dasar dari banyak aplikasi pemrosesan teks, termasuk deteksi kata kunci [1].

Dalam konteks deteksi judul, pola yang dicari adalah daftar kata kunci seperti `slot`, `gacor`, `rtp`, `jackpot`, dan frasa lain yang berkaitan dengan promosi judi daring. Pencocokan dapat dilakukan secara eksak maupun tidak eksak. Pencocokan eksak hanya menerima string yang sama persis, sedangkan pencocokan tidak eksak menerima string yang mirip berdasarkan ukuran tertentu.

## 2.4 Regular Expression

*Regular expression* (RegEx) adalah notasi untuk mendeskripsikan pola teks. RegEx memungkinkan pencarian tidak hanya berdasarkan kata tetap, tetapi juga berdasarkan struktur karakter tertentu [2]. Contohnya, pola huruf yang diikuti angka dapat digunakan untuk mendeteksi nama situs atau kode promosi yang ditulis seperti `SLOT88`, `ZEUS222`, atau `MAXWIN234`.

Pada Judol Detector, RegEx digunakan untuk mendeteksi pola kata yang terdiri atas minimal dua huruf dan diikuti dua hingga tiga angka. Pola ini menangkap format umum yang sering muncul pada konten promosi judol, terutama ketika nama brand atau istilah promosi digabungkan dengan angka.

Secara konseptual, pola yang digunakan dapat dijelaskan sebagai berikut:

Tabel 2.1: Komponen pola RegEx

| Komponen                    | Makna                                                                                          |
|-----------------------------|------------------------------------------------------------------------------------------------|
| Batas kiri token            | Memastikan pola tidak berada di tengah kata lain.                                              |
| Huruf minimal dua karakter  | Menangkap bagian nama atau istilah promosi.                                                    |
| Angka dua sampai tiga digit | Menangkap akhiran numerik seperti <code>88</code> , <code>777</code> , atau <code>222</code> . |
| Batas kanan token           | Memastikan pola berakhir sebagai token utuh.                                                   |



## 2.5 Levenshtein Distance

*Levenshtein Distance* adalah ukuran jarak antara dua string berdasarkan jumlah minimum operasi edit yang diperlukan untuk mengubah satu string menjadi string lain [3]. Operasi edit yang umum digunakan adalah:

1. penyisipan karakter,
2. penghapusan karakter,
3. penggantian karakter.

Jika jarak edit antara dua string kecil, maka kedua string dianggap mirip. Sebagai contoh, kata **gacor** dan **g4cor** hanya berbeda pada satu karakter jika angka 4 dianggap sebagai pengganti huruf a.

### 2.5.1 Weighted Levenshtein Distance

Weighted Levenshtein Distance adalah variasi Levenshtein Distance yang memberi bobot berbeda pada operasi edit tertentu. Pada Judol Detector, penggantian karakter yang mirip secara visual diberi biaya lebih kecil dibandingkan penggantian karakter biasa. Pendekatan ini digunakan karena banyak konten judol memodifikasi kata dengan karakter yang tampak mirip.

Tabel 2.2: Contoh karakter yang dianggap mirip secara visual

| Huruf | Variasi visual |
|-------|----------------|
| o     | 0              |
| a     | 4              |
| i     | 1, l,          |
| e     | 3              |
| s     | 5, \$          |
| b     | 8              |
| g     | 6, 9           |
| t     | 7              |

Nilai kemiripan dihitung dari jarak edit berbobot. Jika nilai kemiripan melebihi ambang batas tertentu, token pada halaman dianggap cocok dengan kata kunci. Pada implementasi program, ambang batas default yang digunakan adalah 0,78. Nilai ini dipilih agar sistem cukup sensitif terhadap variasi penulisan, tetapi tetap membatasi jumlah hasil positif palsu.

## 2.6 Boyer-Moore

Algoritma Boyer-Moore adalah salah satu algoritma pencocokan string yang bekerja dengan membandingkan karakter dari kanan ke kiri. Inti dari algoritma ini adalah menggunakan *bad character heuristic* untuk menentukan seberapa jauh pola digeser ketika terjadi ketidakcocokan.

Inti dari *bad character heuristic* cukup simpel. Ketika kita membandingkan pola dengan teks dan menemukan karakter yang tidak cocok, karakter di teks tersebut disebut *bad character*. Begitu terjadi ketidakcocokan, pola tidak digeser sedikit demi sedikit, melainkan langsung “dilompatkan” sejauh mungkin. Ada dua kemungkinan pergeseran: pertama, pola digeser hingga karakter buruk di teks bisa cocok dengan kemunculan terakhirnya di dalam pola; kedua, jika karakter tersebut sama sekali tidak ada di pola, maka pola langsung dilewati melewati karakter yang tidak cocok itu. [7]

Dengan heuristic ini, algoritma dapat melompati sejumlah posisi yang tidak perlu diperiksa, sehingga pada kasus terbaik kompleksitasnya mendekati  $O(n/m)$ .

### 2.6.1 Algoritma Rabin-Karp

Berbeda dengan algoritma pencocokan string yang membandingkan karakter satu per satu secara langsung, algoritma Rabin-Karp menggunakan fungsi hash untuk mempercepat pencarian. Ide utamanya adalah menghitung nilai hash dari pola, lalu membandingkannya dengan hash setiap jendela teks yang berukuran sama dengan pola. Jika hash cocok, barulah dilakukan pemeriksaan karakter demi karakter untuk memastikan tidak terjadi *hash collision* [8].

Untuk menghitung hash, setiap karakter dianggap sebagai digit dalam sistem bilangan berbasis  $d$  (biasanya 256 untuk ASCII), lalu nilai tersebut diambil modulo sebuah bilangan prima  $q$ . Modulo ini berfungsi untuk mencegah *overflow* dan mengurangi kemungkinan tabrakan hash. Agar tidak perlu menghitung ulang dari nol setiap kali jendela bergeser, algoritma ini menggunakan teknik *rolling hash*: karakter paling kiri dikeluarkan, sisa nilai dikalikan ulang dengan basis, lalu karakter baru di kanan dimasukkan. Dengan cara ini, hash jendela berikutnya bisa diperoleh dalam waktu konstan.

Pada kasus rata-rata, algoritma ini berjalan dengan kompleksitas  $O(n + m)$  karena setiap pergeseran hanya melakukan satu kali operasi aritmetika. Namun, pada kasus terburuk ketika banyak tabrakan hash terjadi, kompleksitasnya bisa naik menjadi  $O(n \times m)$  karena setiap tabrakan harus diverifikasi secara manual karakter per karakter.

## 2.7 Tokenisasi Teks

Sebelum pencocokan fuzzy dilakukan, teks perlu dipecah menjadi kandidat token. Token adalah potongan teks yang berisi huruf, angka, garis bawah, atau beberapa simbol yang relevan. Tokenisasi memungkinkan program membandingkan kata kunci dengan bagian teks yang berukuran sebanding.

Untuk kata kunci yang terdiri atas beberapa kata, seperti `slot gacor` atau `bonus deposit`, program membentuk jendela token dengan jumlah token yang sama dengan jumlah token pada kata kunci. Dengan cara ini, frasa multi-kata tetap dapat dibandingkan menggunakan Weighted Levenshtein.

## 2.8 Deduplication dan Cache

Pipeline deteksi dapat menghasilkan lebih dari satu hasil pada rentang teks yang sama, misalnya ketika satu token cocok oleh RegEx dan juga cocok oleh Weighted Levenshtein. Oleh karena itu, program melakukan deduplikasi berdasarkan rentang indeks awal dan akhir dari hasil deteksi. Hasil yang memiliki rentang sama hanya disimpan sekali.

Program juga menggunakan cache untuk menyimpan hasil pemindaian teks. Cache menghindari penghitungan ulang terhadap teks, daftar kata kunci, dan opsi deteksi yang sama. Dengan demikian, pemindaian berulang pada halaman yang sama dapat dilakukan lebih efisien.

## 2.9 Optical Character Recognition

*Optical Character Recognition* (OCR) adalah proses mengenali teks dari gambar. OCR berguna ketika teks tidak tersedia sebagai node DOM, melainkan menjadi bagian dari banner, poster, atau gambar promosi. Pada Judol Detector, OCR dilakukan menggunakan Tesseract.js, yaitu port JavaScript dari mesin OCR Tesseract [4].

Proses OCR pada program terdiri atas beberapa tahap:

1. memilih elemen gambar yang terlihat pada viewport;
2. menggambar gambar ke canvas agar pikselnya dapat diproses;
3. melakukan preprocessing citra sebelum OCR;
4. menjalankan OCR dengan Tesseract.js pada gambar hasil preprocessing;
5. menjalankan pipeline deteksi pada teks hasil OCR;
6. memberi penanda dan blur pada gambar yang dinyatakan positif.

Preprocessing dilakukan agar masukan yang diberikan ke Tesseract lebih dekat dengan bentuk citra yang mudah dikenali, yaitu teks gelap pada latar terang. Tahap preprocessing yang digunakan meliputi penskalaan gambar, penambahan border putih, penggabungan kanal transparansi dengan latar putih, konversi ke grayscale, peningkatan kontras, inversi warna apabila gambar cenderung gelap, serta binarisasi adaptif berdasarkan rata-rata lokal di sekitar setiap piksel. Pendekatan threshold lokal digunakan karena gambar promosi sering memiliki background warna-warni sehingga satu ambang global tidak selalu cukup stabil.

## 2.10 Highlight, Tooltip, dan Blur

Setelah hasil deteksi ditemukan, program perlu menyampaikan informasi tersebut kepada pengguna. Judol Detector melakukan hal ini dengan memodifikasi DOM

halaman. Teks yang terdeteksi dibungkus dalam elemen `span` dengan class khusus. Class tersebut diberi style agar teks terlihat sebagai highlight.

Setiap highlight menyimpan metadata deteksi pada atribut `data-*`, yaitu kata kunci, algoritma, jumlah hasil, dan waktu eksekusi. Metadata ini digunakan untuk menampilkan tooltip ketika pengguna mengarahkan kursor ke hasil deteksi.

Fitur blur digunakan untuk menyamarkan konten yang terdeteksi. Pada teks, blur dapat diaktifkan atau dimatikan melalui popup. Pada gambar hasil OCR yang positif, blur diberikan secara otomatis agar konten visual yang dicurigai tidak langsung terlihat jelas.

## 2.11 Kompleksitas Algoritma

Analisis kompleksitas digunakan untuk memperkirakan biaya komputasi pipeline deteksi. Jika  $n$  adalah panjang teks,  $k$  adalah jumlah kata kunci, dan  $m$  adalah panjang rata-rata kata kunci, maka:

- RegEx berjalan terhadap teks dengan biaya yang bergantung pada engine RegEx dan pola yang digunakan, tetapi pada pola sederhana umumnya linear terhadap panjang teks.
- Weighted Levenshtein membandingkan kandidat token dengan kata kunci. Setiap perbandingan dua string membutuhkan  $O(m^2)$  pada kasus umum karena menggunakan tabel dinamis dua dimensi.
- OCR memiliki biaya paling besar karena melibatkan pengenalan citra. Oleh karena itu, jumlah gambar yang diproses dibatasi dan hasil OCR disimpan dalam cache.

Dalam implementasi, performa dijaga dengan beberapa strategi: membatasi jumlah gambar OCR, menyaring gambar berdasarkan ukuran dan visibilitas, menggunakan cache hasil teks dan OCR, serta melakukan deduplikasi hasil deteksi agar output tidak berulang.

## Bab 3

# Analisis Pemecahan Masalah

### 3.1 Langkah-Langkah Pemecahan Masalah

Pengembangan Judol Detector diawali dari identifikasi masalah utama: **bagaimana mendeteksi indikasi konten judi online pada halaman web secara otomatis dan efisien?** Masalah ini dipecah menjadi lima langkah sistematis berikut.

1. **Identifikasi pola dan variasi teks.** Konten judi online tidak selalu ditulis secara eksplisit. Selain kata kunci tetap seperti `slot`, `gacor`, dan `jackpot`, terdapat pola khusus (misalnya `SLOT88`, `MAXWIN234`) serta variasi penulisan dengan karakter mirip secara visual (misalnya `G4C0R`, `H0KI88`).
2. **Pemilihan strategi pencocokan string.** Untuk kata kunci tetap, dipilih algoritma exact matching: KMP dan Boyer-Moore, Rabin-Karp, dan Aho-Corasick. Untuk pola dengan variasi visual, digunakan Regular Expression dan Weighted Levenshtein Distance.
3. **Perancangan mekanisme pemindaian halaman.** Sistem harus mampu membaca teks dari DOM halaman web, mengabaikan elemen yang tidak relevan (`script`, `style`, `input`), serta memproses teks dari gambar melalui OCR.
4. **Penyajian hasil kepada pengguna.** Hasil deteksi perlu ditampilkan secara intuitif melalui highlight pada elemen DOM, tooltip saat hover, dan ringkasan statistik pada popup extension.
5. **Optimasi performa.** Untuk menghindari pemrosesan berulang, diterapkan mekanisme deduplikasi hasil antar-algoritma dan penyimpanan cache pada pemindaian teks dan OCR.

### 3.2 Pemetaan Masalah menjadi Elemen Algoritma

Setiap aspek masalah yang telah diidentifikasi dipetakan ke elemen algoritma atau modul yang bertanggung jawab menyelesaikannya. Pemetaan ini memperjelas peran masing-masing algoritma dalam sistem secara keseluruhan.

Tabel 3.1: Pemetaan masalah ke elemen algoritma

| Aspek Masalah                                                                 | Elemen Algoritma / Modul                                                                                                  |
|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Kata kunci tetap dari <code>keyword.txt</code>                                | KMP (failure function), Boyer-Moore (bad character table), Rabin-Karp (rolling hash), Aho-Corasick (trie & failure links) |
| Pola <code>&lt;kata&gt;&lt;angka&gt;</code> (contoh: <code>MAXWIN234</code> ) | RegEx engine JavaScript dengan pola <code>[\p{L}]{2,}[0-9]{2,3}</code>                                                    |
| Variasi penulisan visual (contoh: <code>G4C0R</code> , <code>H0KI88</code> )  | Weighted Levenshtein Distance dengan biaya substitusi karakter mirip yang lebih rendah                                    |
| Pemrosesan teks dari gambar                                                   | OCR Tesseract.js → pipeline pencocokan string                                                                             |
| Penggabungan hasil berbagai algoritma                                         | Modul <code>scanner.ts</code> : normalisasi, deduplikasi, cache LRU                                                       |
| Penandaan hasil pada halaman web                                              | DOM manipulation: wrapper <code>&lt;span&gt;</code> , tooltip, blur                                                       |

**Pemetaan ke KMP.** Masalah yang dihadapi adalah mencari kemunculan beberapa pola (keyword) dalam teks yang panjang dan dinamis (DOM halaman web). KMP dipilih karena setiap kali terjadi ketidakcocokan, informasi yang sudah cocok sebelumnya tidak dibuang. Failure function memungkinkan pola digeser tepat ke posisi di mana prefiks yang sudah cocok bisa dimanfaatkan kembali. Hal ini sangat cocok untuk teks halaman web yang seringkali mengandung pengulangan huruf atau frasa, sehingga jumlah perbandingan karakter tetap linear.

**Pemetaan ke Boyer-Moore.** BM dipilih karena keunggulannya pada kasus teks alfanumerik yang umum di halaman web. Ketika karakter pada teks tidak ditemukan dalam pola sama sekali, BM dapat melompati sebagian besar teks tanpa perlu membandingkan karakter satu per satu. Karakteristik ini sangat menguntungkan pada pencarian keyword pendek (misalnya `slot`, `rtp`) dalam teks halaman yang panjang, karena jumlah pergeseran bisa sangat besar.

**Pemetaan ke Rabin-Karp.** RK dipilih karena kemampuannya mencari banyak pola sekaligus melalui pendekatan rolling hash. Setiap keyword dihitung nilai hash-nya terlebih dahulu; kemudian window hash pada teks digeser satu karakter per langkah. Ketika nilai hash window cocok dengan hash pola, verifikasi karakter dilakukan untuk menghindari hash collision. Pendekatan ini efisien ketika banyak keyword pendek perlu ditemukan dalam teks yang sama karena perbandingan hash jauh lebih cepat daripada perbandingan karakter penuh.

**Pemetaan ke Aho-Corasick.** Aho-Corasick dipilih untuk skenario pencarian banyak keyword secara simultan. Semua keyword dikompilasi menjadi trie dengan failure links, sehingga teks hanya perlu dilintasi satu kali untuk menemukan seluruh kemunculan semua keyword sekaligus. Berbeda dengan KMP dan BM yang memproses satu keyword per lintasan, Aho-Corasick menyelesaikan seluruh daftar keyword dalam satu traversal linear, menjadikannya sangat efisien untuk keyword list yang panjang.

## 3.3 Fitur Fungsional dan Arsitektur Extension

### 3.3.1 Fitur Fungsional

Judol Detector menyediakan fitur-fungsional berikut kepada pengguna.

- **Pemindaian teks halaman.** Content script membaca seluruh teks yang tampil pada DOM menggunakan `TreeWalker`, mengabaikan elemen `script`, `style`, `noscript`, `textarea`, dan `input` agar pemindaian fokus pada konten yang benar-benar dibaca pengguna.
- **Pemindaian multi-algoritma.** Enam algoritma dijalankan secara independen pada setiap node teks: KMP, Boyer-Moore, Rabin-Karp, Aho-Corasick, Regular Expression, dan Weighted Levenshtein Distance.
- **Highlight dan tooltip.** Teks yang terdeteksi dibungkus dalam elemen `<span>` dengan class khusus. Tooltip muncul saat pengguna mengarahkan kursor, menampilkan keyword, algoritma asal, jumlah kemunculan, dan waktu eksekusi.
- **Blur otomatis.** Pengguna dapat mengaktifkan efek blur pada teks terdeteksi melalui popup extension. Gambar hasil OCR yang dinyatakan positif akan diburamkan secara otomatis.
- **OCR gambar.** Gambar yang terlihat pada viewport diproses langsung menggunakan `Tesseract.js`. Sistem tidak menggunakan metadata gambar seperti nama file, `alt`, `aria-label`, atau teks container sebagai sumber deteksi, sehingga keputusan positif berasal dari teks yang berhasil diekstraksi melalui OCR.
- **Rescan dan cache.** Tombol rescan pada popup menghapus highlight lama dan menjalankan pipeline deteksi dari awal. Hasil pemindaian disimpan dalam cache LRU agar pemrosesan berulang pada teks yang sama menjadi instan.

### 3.3.2 Arsitektur Extension

Sistem dibangun dengan tiga komponen utama yang saling berkomunikasi.

1. **Content Script** (`src/content/`). Bertugas menelusuri DOM, menjalankan `scanTextForJudol`, memasang highlight, tooltip, efek blur, serta menangani pesan dari popup melalui Chrome Messaging API.
2. **Popup** (`src/popup/`). Antarmuka berbasis React yang menampilkan statistik real-time: total match, waktu eksekusi per algoritma, distribusi keyword, dan kontrol toggle (blur, OCR).
3. **Modul Algoritma** (`src/algorithms/`). Kumpulan fungsi murni yang menerima (`text`, `keywords`) dan mengembalikan `AlgorithmResult`. Modul ini tidak bergantung pada API browser sehingga dapat diuji secara mandiri melalui unit test.



### 3.4 Contoh Ilustrasi Kasus

Berikut adalah ilustrasi bagaimana pipeline deteksi bekerja pada sebuah potongan teks halaman web.

**Teks halaman:**

Daftar sekarang di GACOR99 dan dapatkan MAXWIN234!

Main slot gacor hari ini. RTP tinggi, jackpot menanti.

**Daftar keyword (keyword.txt):** slot, gacor, jackpot, rtp, bonus deposit  
**Langkah deteksi:**

1. **KMP** menemukan "slot" pada indeks 56–59, "gacor" pada indeks 61–65, "rtp" pada indeks 77–79, dan "jackpot" pada indeks 89–95.
2. **Boyer-Moore** menemukan hasil yang sama pada posisi identik untuk keempat keyword tersebut.
3. **Rabin-Karp** menghasilkan hash match untuk "slot", "gacor", "rtp", dan "jackpot", lalu memverifikasi karakter demi karakter hingga dinyatakan cocok.
4. **Aho-Corasick** menelusuri teks satu kali menggunakan trie yang telah dikompilasi dari seluruh keyword, dan menemukan keempat keyword yang sama pada posisi yang identik.
5. **RegEx** menemukan pola GACOR99 (indeks 19–25) dan MAXWIN234 (indeks 40–48) karena keduanya terdiri atas minimal dua huruf dan diikuti dua hingga tiga angka tanpa spasi.
6. **Weighted Levenshtein** tidak menemukan kecocokan baru karena semua keyword pada daftar sudah terdeteksi secara eksak oleh algoritma sebelumnya.

**Hasil setelah deduplikasi:**

- GACOR99 — deteksi oleh RegEx
- MAXWIN234 — deteksi oleh RegEx
- slot — deteksi oleh KMP / Boyer-Moore / Rabin-Karp / Aho-Corasick (diambil satu karena rentang sama)
- gacor — deteksi oleh KMP / Boyer-Moore / Rabin-Karp / Aho-Corasick (diambil satu karena rentang sama)
- rtp — deteksi oleh KMP / Boyer-Moore / Rabin-Karp / Aho-Corasick (diambil satu karena rentang sama)
- jackpot — deteksi oleh KMP / Boyer-Moore / Rabin-Karp / Aho-Corasick (diambil satu karena rentang sama)

Setelah deduplikasi, keenam hasil tersebut di-highlight pada DOM dan ringkasan statistiknya ditampilkan pada popup extension.





## Bab 4

# Implementasi dan pengujian



## Bab 5

## Penutup



## Bab 6

## Lampiran

# Bibliografi

- [1] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [2] J. E. F. Friedl, *Mastering Regular Expressions*, 3rd ed. O'Reilly Media, 2006.
- [3] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [4] Tesseract.js Contributors, "Tesseract.js documentation," <https://tesseract.projectnaptha.com/>, 2026, accessed: May-2026.
- [5] Chrome Developers, "Chrome extensions documentation," <https://developer.chrome.com/docs/extensions/>, 2026, accessed: May-2026.
- [6] MDN Web Docs, "Document object model (DOM)," [https://developer.mozilla.org/en-US/docs/Web/API/Document\\_Object\\_Model](https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model), 2026, accessed: May-2026.
- [7] GeeksforGeeks, "Boyer moore algorithm for pattern searching," 2025, accessed: May-2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/>
- [8] —, "Rabin-karp algorithm for pattern searching," 2026, accessed: May-2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/rabin-karp-algorithm-for-pattern-searching/>